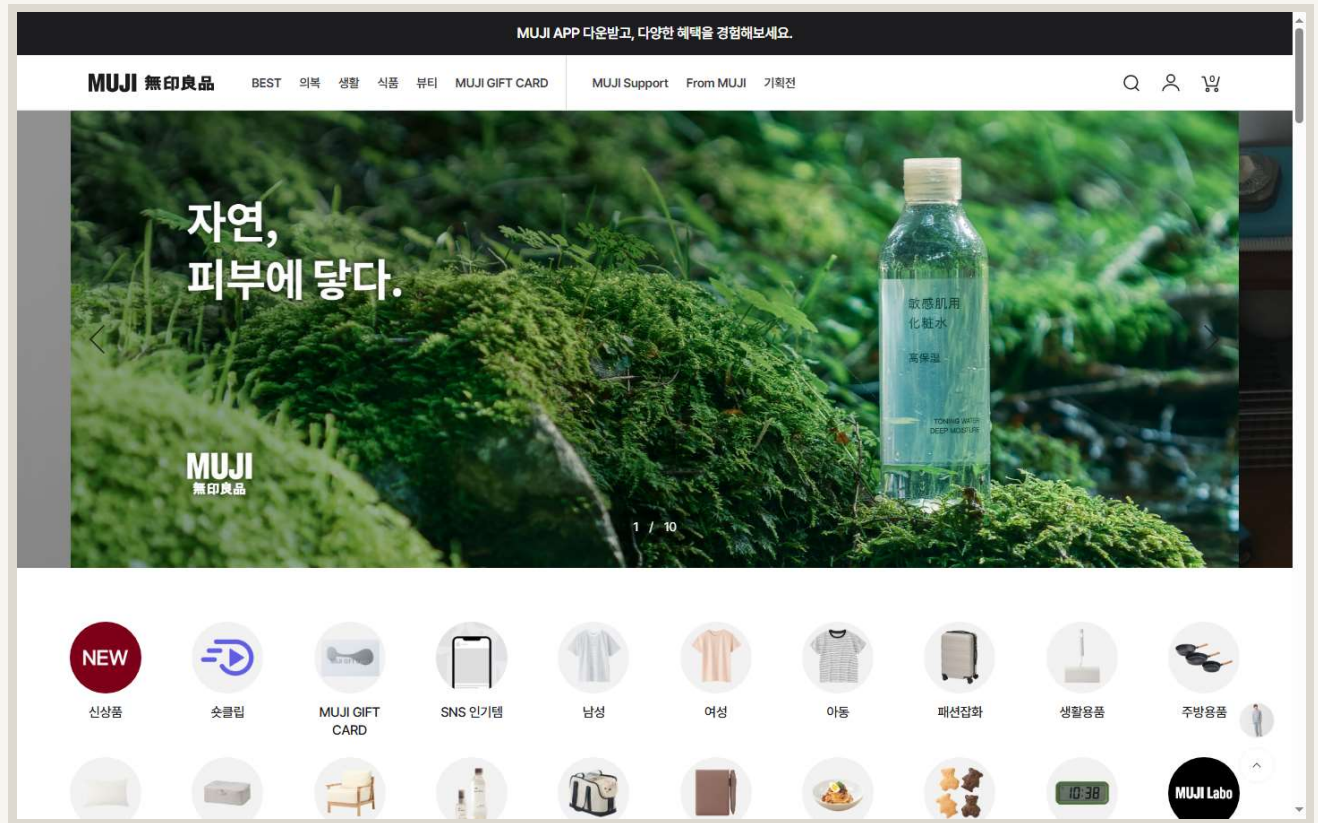


MUJI CLONE

React로 설계한 데이터 흐름과 상태 관리

라우팅 · JSON 렌더링 · 상태 설계 · 예외 처리
코드 근거와 검증 결과 중심



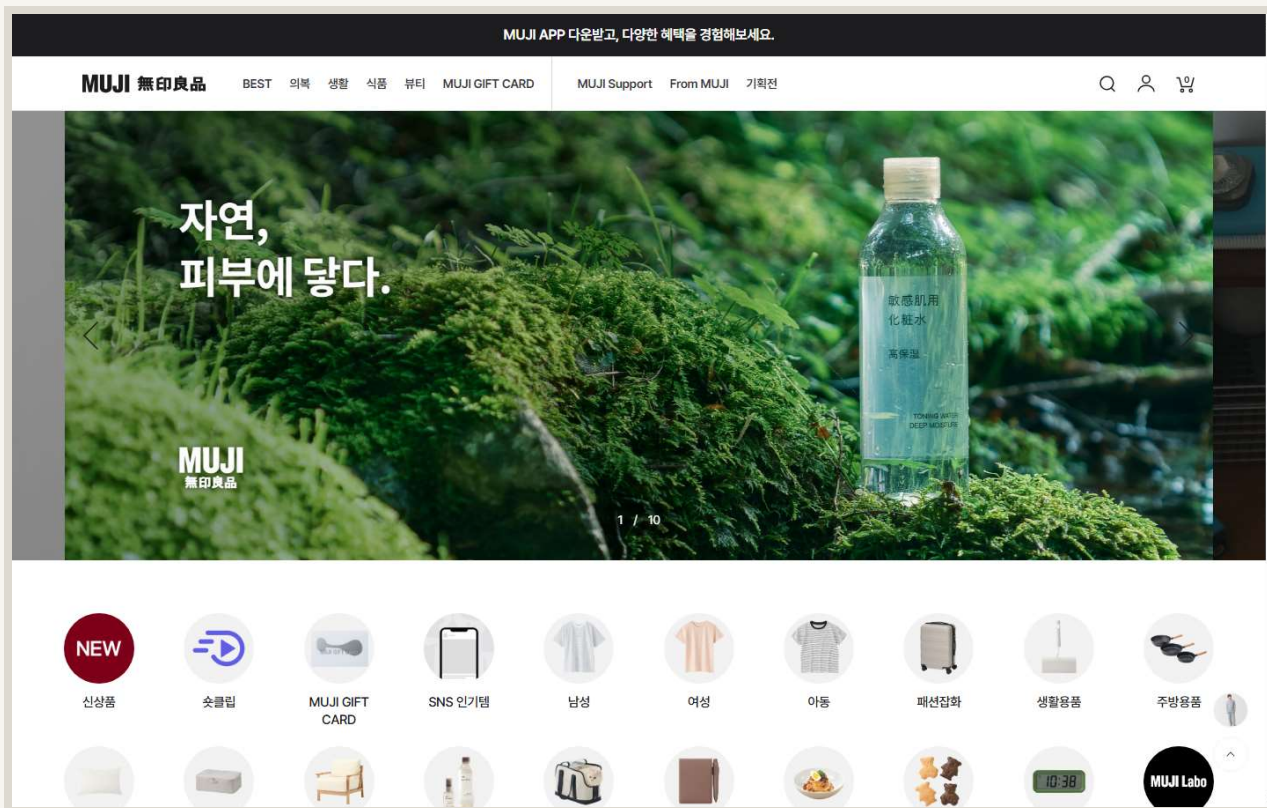
무인양품 코리아를 참고한 학습용 프론트엔드 클론

RESPONSIVE / PC + MOBILE

메인 — 기기별 데이터와 배치를 한 컴포넌트에서 분리

mainBannersPC · 데스크톱 메뉴 / mainBannersMO · fixed 하단 메뉴

PC · 1440 × 900



PC
↓
모바일

MOBILE · 390 × 844

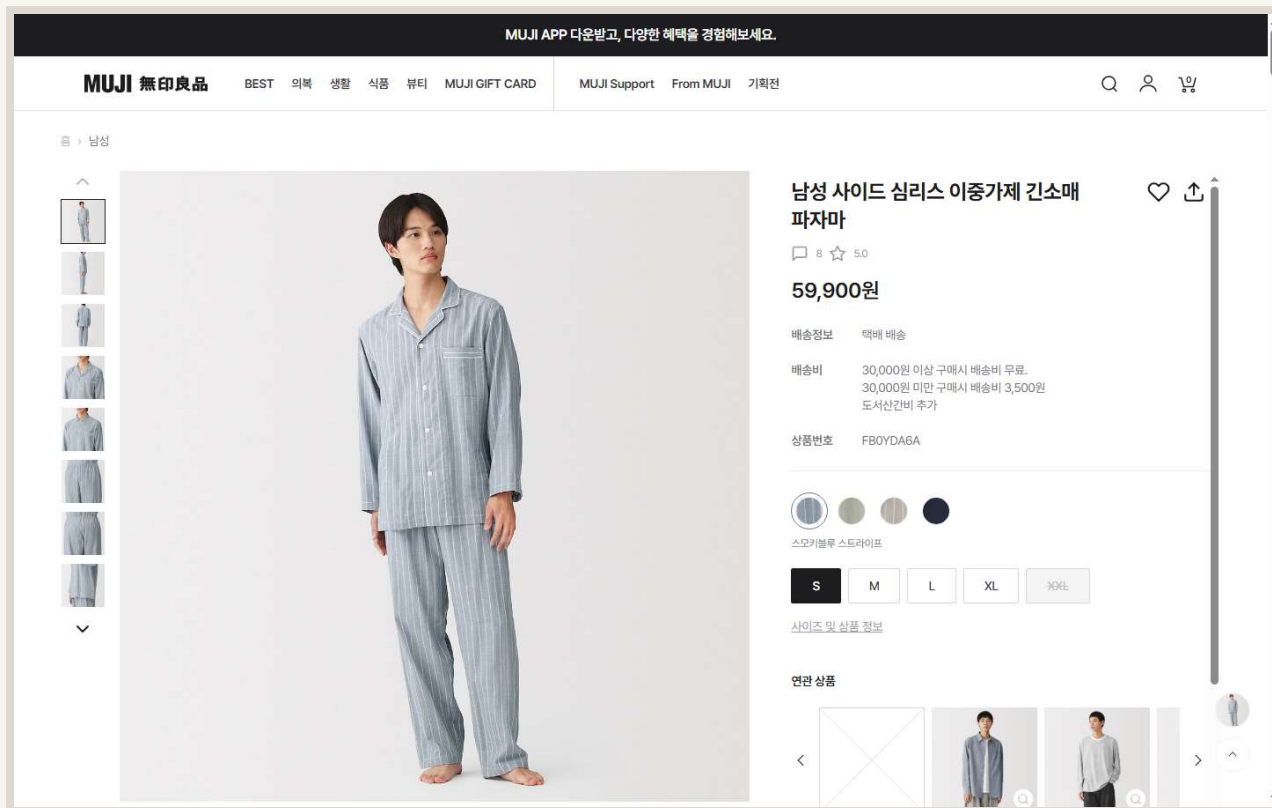


RESPONSIVE / PC + MOBILE

상품 상세 — 공통 상태와 breakpoint로 화면을 동기화

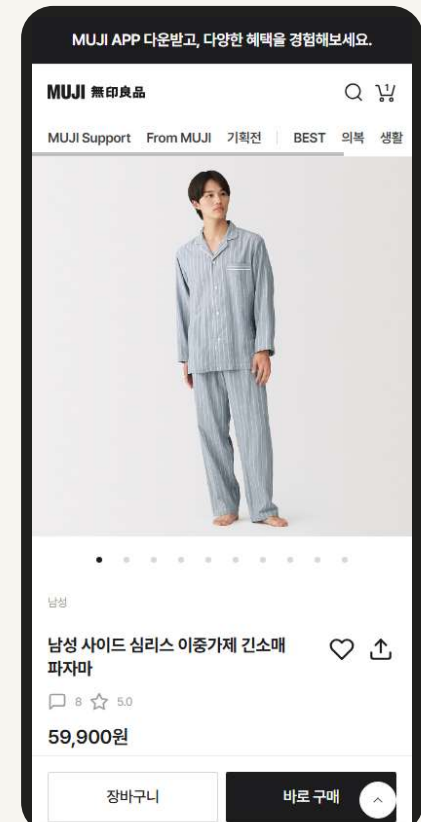
Detail → Gallery + ProductInfo / Swiper direction · CSS Grid 분기

PC · 1440 × 900



PC
↓
모바일

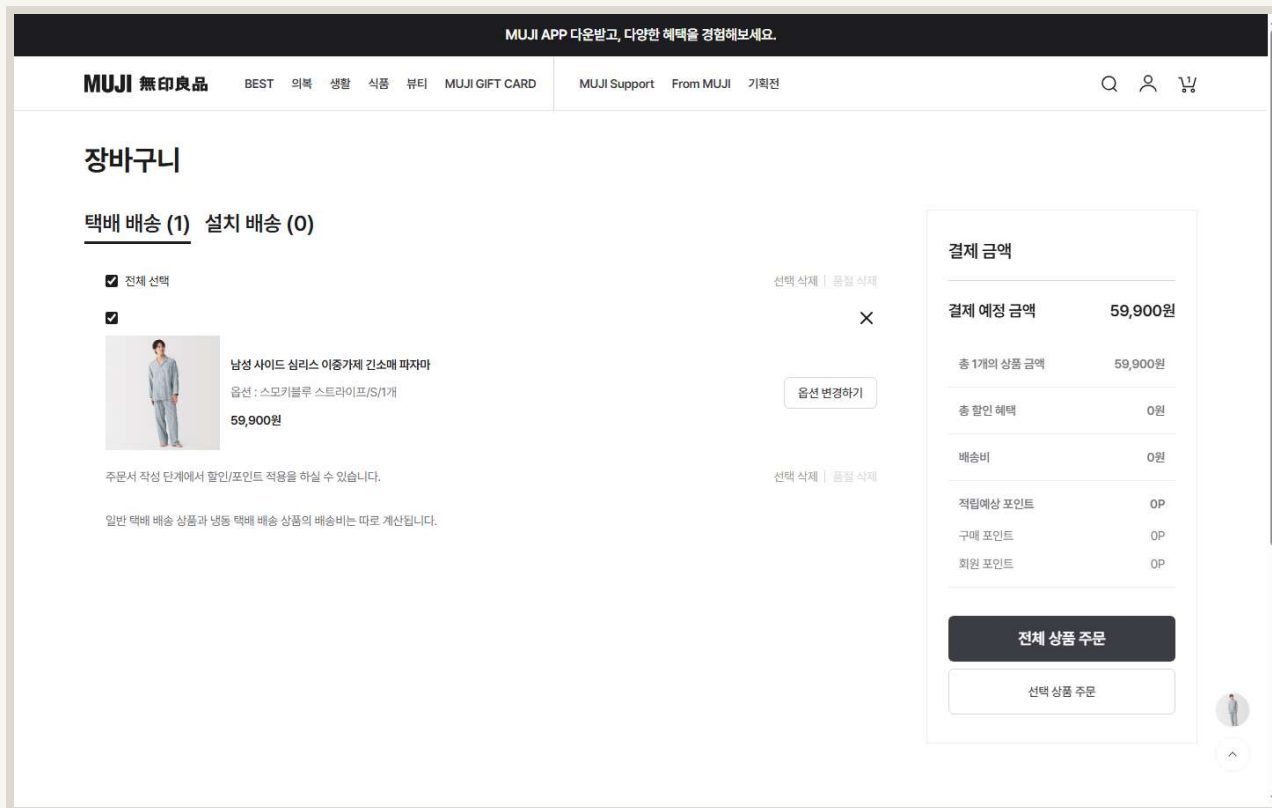
MOBILE · 390 × 844



장바구니 — 같은 Redux 상태를 기기별 UI에 투영

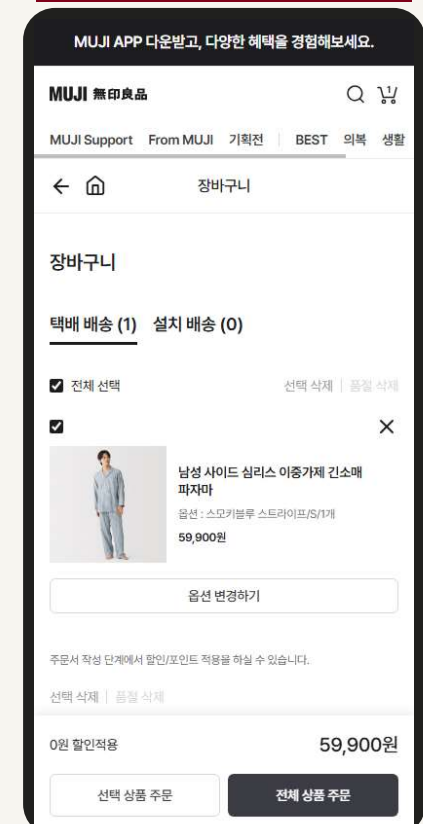
items → getCartLines → totals / 동일 상태 · 하단 고정 요약

PC · 1440 × 900



PC
↓
모바일

MOBILE · 390 × 844



전체 구현을 여섯 개의 흐름으로

각 화면의 모양보다 URL·데이터·상태·저장소가 연결되는 순서를 기준으로 설명합니다.

01 ROUTING HashRouter + Routes

URL의 ID를 읽어 Detail·PlanDetail로 연결

02 DATA 정적 JSON → data.js

상품·옵션·배너를 조회 가능한 형태로 정리

03 LOCAL STATE useState + props

색상·옵션·수량처럼 화면 안의 선택을 관리

04 SHARED STATE Redux cart.items

Header·Detail·Cart가 같은 장바구니를 공유

05 PERSISTENCE localStorage

저장 값을 검사한 뒤 Redux 초기 상태로 복구

06 DYNAMIC VIEW pc_content HTML·CSS

하나의 PlanDetail에서 기획전별 구조를 출력

구현 범위: 저장된 데이터 기반 프론트엔드 / 미구현: 인증 · 주문 · 결제 · 실시간 재고 API

화면·데이터·상태의 역할을 구분

저장된 JSON을 읽고, 재사용 가능한 컴포넌트로 화면을 구성합니다.

01

DATA

수집한 JSON

상품 · 옵션 · 이미지
기획전 · 배너

02

VIEW

React 컴포넌트

메인 → 상품 상세
장바구니 · 기획전

03

STATE

사용자 선택

Redux: 공통 장바구니
localStorage: 저장·복구

14

카테고리

288

메인 상품

정적 JSON import

실시간 상품 API 연동은 미구현

2026.09.09 메인 목록 집계 · 기획전 포함 전체 카탈로그 수와 구분

기획전 JSON이 페이지별 레이아웃을 결정

plan_id로 데이터를 찾고, JSON에 저장된 HTML·CSS를 기획전별로 렌더링합니다.

01 ROUTE

기획전 ID

/plans/:id
getPlanDetail(id)



02 JSON

main_plan.json

pc_content
HTML · CSS 문자열



03 RENDER

PlanDetail

이미지 경로 보정
HTML 구조 출력

pc_content 있음

JSON의 HTML · CSS를 그대로 출력

기획전마다 구조와 디자인이 달라짐

pc_content 없음

기본 React 레이아웃으로 분기

일반 목록 또는 일부 기획전 전용 JSX

현재 모바일: mobile_content를 직접 선택하지 않고 pc_content 내부 미디어 쿼리로 대응

상품 ID로 찾고, 색상 상태로 함께 변경

같은 상세 페이지가 다른 상품을 표시하고, 사진과 옵션이 같은 색상 값을 사용합니다.

주소의 상품 ID

`/products/view/1005355`

`useParams` → `getProductDetail(id)`

JSON에서 조회한 상품을
사진·옵션·설명에 props로 전달

상품이 없으면 안내 화면
상품이 바뀌면 `key={id}`로 내부 상태 초기화

부모 Detail: 선택 색상 color

`onColorChange(index)`로 부모 상태 변경

사진 갤러리

선택 색상의 images 표시

상품 옵션

사이즈 해제 · 수량 1로

상태를 공통 부모로 올려
서로 다른 화면 조각의 선택을 맞춥니다.

상태는 사용하는 범위에 따라 소유 위치를 결정

값을 어디에 둘지는 편의가 아니라 누가 읽고 변경하는지에 따라 결정했습니다.

LOCAL / Detail

color

Gallery와 ProductInfo가 함께 사용

공통 부모에서 관리
onColorChange로 변경

LOCAL / ProductInfo

optionId · quantity

구매 옵션 영역 안에서만 사용

색상 변경 시 옵션 해제
수량을 1로 초기화

GLOBAL / Redux

cart.items

Header·Detail·Cart가 함께 사용

최소 ID와 수량만 저장
화면별 표시 값은 다시 계산

색상 변경 규칙 `setColor` → `optionId=''` → `quantity=1` → Gallery key 재생성

이유: 다른 색상에서 선택한 품절·재고·사이즈 조건이 새 색상에 그대로 남는 오류를 방지

장바구니 상태를 공유하고 저장

담기 → 유효성 검사 → 상태 갱신 → 화면 반영 → 저장 순서로 연결합니다.

01

선택·담기

상품 ID
옵션 ID · 수량

02

Redux 검사

동일 옵션 합산
재고 상한 적용

03

화면 갱신

헤더 수량
선택 상품 금액

04

저장·복구

localStorage
앱 시작 시 검사

장바구니가 저장하는 값

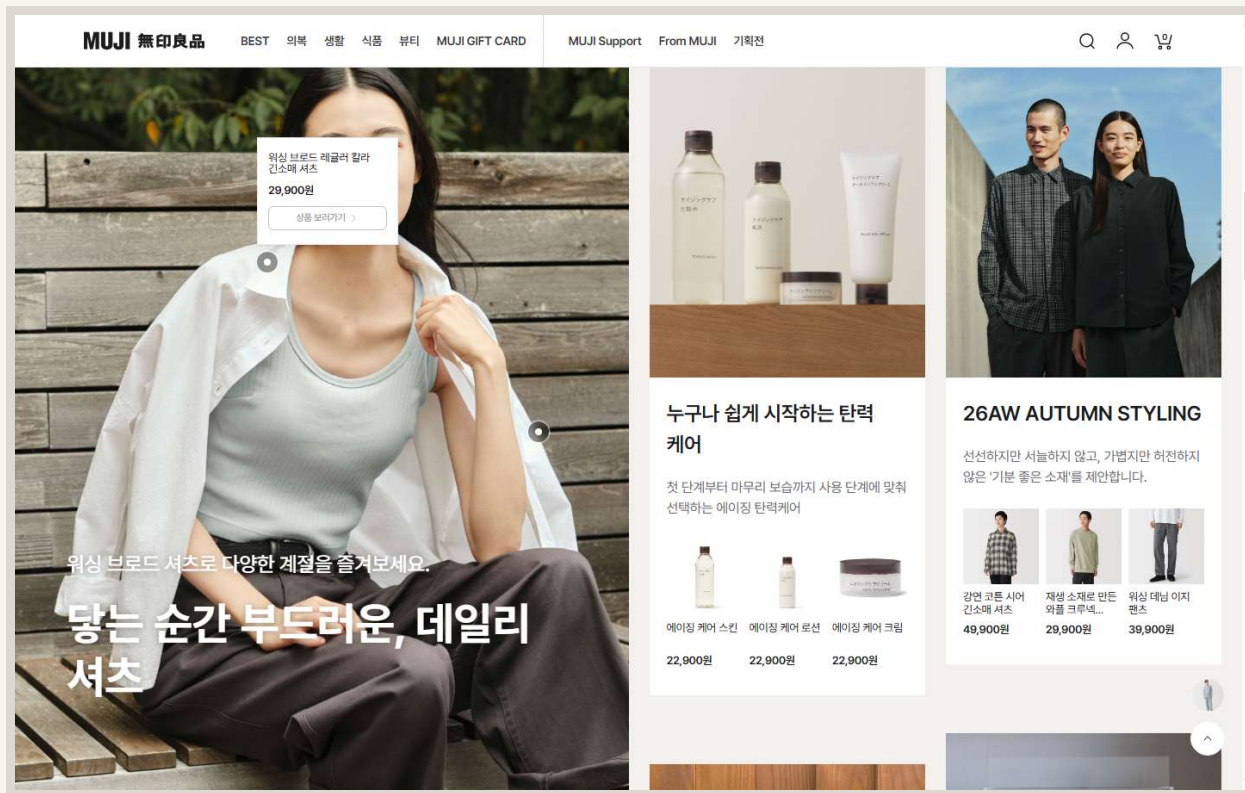
```
{ productId, optionId,  
  quantity, selected }
```

같은 옵션 1개 + 2개 → 3개

상품명·가격은 공통 데이터에서 조회
합계는 선택된 구매 가능 항목으로 계산
새로고침 후 유효한 항목만 복구

화보 고정과 상품 정보를 따로 제어

CSS는 배치와 고정을, React 상태는 정보창의 열림을 담당합니다.



01 화보 고정

sticky · top: 0

PC에서 부모 구역 안에 고정

02 상품 점 배치

JSON의 x-y 좌표를 %로 사용

사진 영역 안에 absolute 배치

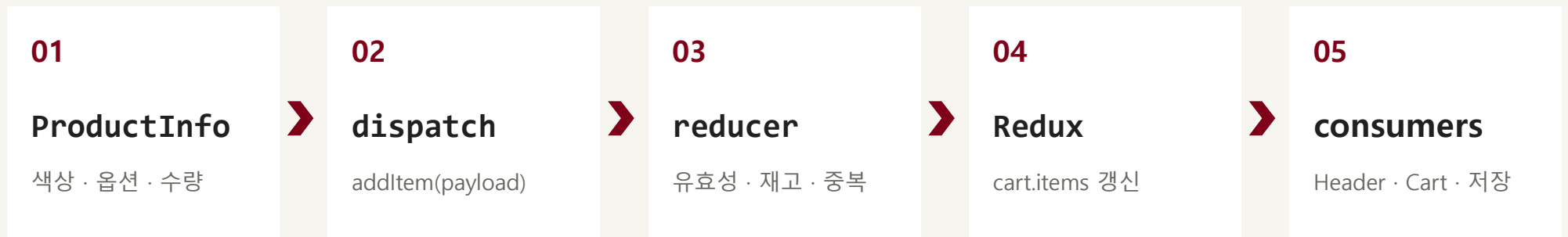
03 열림 상태 관리

active 번호의 정보창만 표시

바깥 클릭·Escape로 닫기

장바구니 담기 1회를 코드 경로로 추적

버튼 클릭부터 저장과 재렌더링까지, 값이 생성·검사·소비되는 위치를 연결합니다.



저장하는 값

```
{ productId, optionId,  
  quantity, selected }
```

지켜야 할 조건

존재하는 옵션만 · 수량 1 이상
재고 상한 적용 · 동일 옵션 합산
품절 항목은 결제 금액에서 제외

store.subscribe → localStorage 저장 / 재실행 시 normalizeCartItems → preloadedState 복구

예외 상황에서는 추측하지 않고 안전한 상태로 분기

정상 흐름뿐 아니라 입력·저장소·콘텐츠가 불완전할 때의 결과를 코드에 정했습니다.

알 수 없는 상품 ID	다른 상품으로 대체하지 않음	"상품을 찾을 수 없습니다" 화면
깨진 localStorage JSON	파싱 오류를 catch	빈 장바구니로 시작
유효하지 않은 저장 항목	옵션 존재·수량·재고 검사	제거 또는 재고 상한으로 보정
저장소 접근 차단	읽기·쓰기를 try/catch	현재 탭 Redux 상태는 계속 사용
pc_content 없음	일반 레이아웃 또는 전용 JSX	기획전 페이지를 fallback 처리
HTML 이미지 절대경로	asset()으로 기준 경로 보정	정적 호스팅 하위 경로 대응

Trade-off 정적 JSON: 재현성 ↑ / 실시간성 ↓ · HTML 문자열: 레이아웃 자유도 ↑ / 외부 입력 보안 검증 필요

배포 이미지 문제를 경로와 용량으로 나눠 해결

증상 → 원인 → 수정 → 확인의 순서로 문제를 설명합니다.

01 / PATH

파일은 있는데 이미지가 안 보임

폴더 변경과 배포 기준 경로를 확인
JSON 참조와 실제 파일 위치를 맞춤

```
asset('/images/...')  
→ 배포 기준 경로를 붙여 반환
```

02 / FILE SIZE

대용량 상품 사진 축소

94101_01.jpg · Git 변경 기록

변경 전 30.58 MB

변경 후 0.136 MB

파일 크기 비교 · 로딩 시간 개선율은 미측정

검증 결과와 남은 작업을 구분

빌드·단위 테스트·브라우저 확인을 같은 결과로 뭉뚱그리지 않고 각각 기록합니다.

BUILD

프로덕션 빌드 성공 · 종료 코드 0

접근성 링크·미사용 변수·Hook 관련 ESLint 경고가 남음

TEST

store.test.js 통과 · 9개 테스트

App.test.js는 옛 ProductDetail import로 suite 시작 실패

BROWSER

PC·모바일 담기와 복구 확인

2026.09.09 캡처: 같은 옵션 담기 → 새로고침 후 값 유지

다음 작업: App.test.js를 현재 Detail 경로로 갱신 → UI 회귀 테스트 재실행 → 경고와 미연결 기능 정리

AI 활용 범위 및 작업 방식

AI를 보조 도구로 활용하며, 전체적인 설계와 결과물 검증은 직접 수행했습니다.

01 프로젝트 설계 및 구조화

프로젝트 설계

원본 사이트 구조 분석 및 UI 컴포넌트 트리 설계
초기 개발 환경 세팅 및 디렉토리 구조 기준 수립

02 UI/UX 구현 및 문제 해결

구현 및 문제 해결

반복적인 레이아웃 및 반응형 CSS 초안 작성 보조
에러 발생 시 시스템 구조를 바탕으로 한 원인 분석

03 주도권 유지 및 문서화

코드 검증 및 문서화

AI 제안 코드의 동작 원리 파악 및 선별적 적용
가이드 문서(PROJECT_GUIDE)를 통한 전체 작업 방향 유지

작업 흐름 목표 설정 → 구현 방안 도출 → 코드 검증 → 가이드 문서화

요약: 명확한 요구사항 정의와 결과물 검증을 통해 AI를 효과적인 실무 도구로 활용했습니다.

구현 근거

핵심 코드 위치

라우팅과 화면 책임	src/App.js	Route path → Main / Detail / Cart / PlanDetail
기획전 HTML 렌더링	src/pages/PlanDetail.js	pc_content → asset 경로 보정 → HTML 출력
상품·색상 상태	src/pages/Detail.js / ProductInfo.js	useParams · color · optionId · quantity
담기·금액·저장	src/store.js	addItem · getCartTotals · normalize · subscribe
반응형 렌더링	src/components/ProductGallery.js / BottomNav.js	breakpoint direction · fixed + 여백
자동 검증 근거	src/store.test.js / App.test.js	정상·중복·재고·오류·복구 시나리오